

Python on the IBMi- Chapter 8



Steel clashes with steel as our hero squares off against a formidable foe—the dark knight. This enemy doesn't roar or cast spells; he waits patiently, exposing flaws in logic and exploiting every oversight. To win this battle, our hero must fight with precision, not just power.

In this chapter, we'll enable editing of our Product Master records through a modal form. When the edit button is clicked, the modal will load the current data, allow changes, and perform validation to ensure each field is correct before submission. If everything checks out, the update is applied and the table refreshed. But if something is off, an error message will appear because in this fight, details matter.

Disclaimer

This article will use the built-in Python HTTP Server. While this works, it is NOT production ready. While it might work for simple utilities and very low demand. It's a very simple HTTP Server that handles basic requests.

Prerequisites

- Please refer to chapter 1 for other prerequisites (<https://www.linkedin.com/pulse/python-ibmi-chapter-1-ricky-thompson-zg7te/>)
- Code can be found here: (https://github.com/thomprl/ibmi_python_rpg_adventures)

Objective

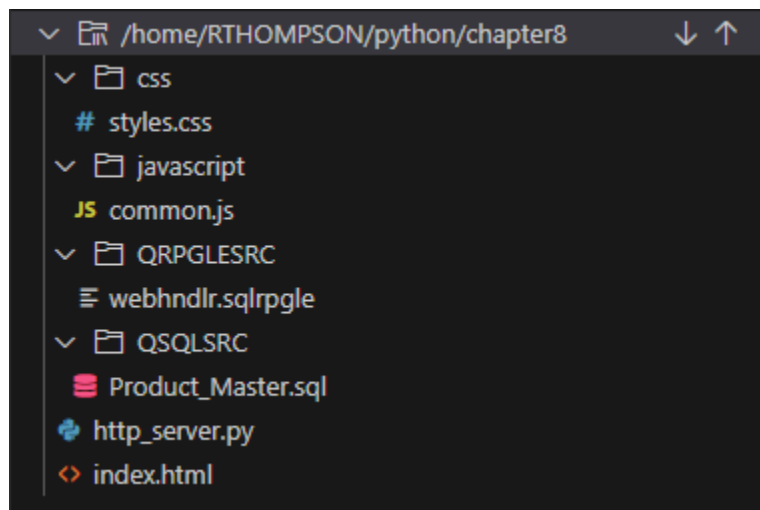
- Add the ability to update and validate a record from our web page

Before We Begin

Again there will be a good amount of code used in this article. Most of it is repeated with updates from the previous chapters but it will be hard to explain in detail what each line of code is doing.

- RPG program (WEBHNDLR) we will add the update and validation/edit operations for our DB2 data/table
- HTML (index.html) several changes to handle the update operation
- The CSS file (**styles.css**) we add styling for the edit modal
- No changes to Python Script (http_server.py) as it will just be forwarding the requests on.
- Changes to our (**common.js**) JavaScript file to handle the update request and changes to the error notification.

This might come in handy as well. Each article I make a new IFS folder and put all the contents into it. This is a picture of my (**/home/rthompson/python/chapter8**) folder from VS Code.



Let's Code

I'm going to repeat most of the code in this article except for the SQL Table creation. If you need that please go to the GitHub site or the previous article. I will repeat the Python script as I think it's important even though there were no changes.

http_server.py

Let's start with the Python script. This Python Script will handle our traffic for both the GET/POST functionality in our application and forward them on to the RPG program.

```
# Import the built-in HTTP server class for handling web
requests
from http.server import HTTPServer, SimpleHTTPRequestHandler,
ThreadingHTTPServer
from urllib.parse import urlparse, parse_qs

# Import argparse for command-line argument parsing
import argparse
# Import standard libraries for handling JSON
import json
import threading

# Import IBM DB2 connection tools and itoolkit
import ibm_db
from itoolkit import *
from itoolkit.transport import DirectTransport

# Create a transport object that allows the toolkit to talk to
IBM i using SQL
itransport = DirectTransport()
print("Starting HTTP Server... Please wait...")

# -----
# DEFINE HTTP REQUEST HANDLER CLASS
# -----

class RPGHTTPRequestHandler(SimpleHTTPRequestHandler):
    LIBRARY = 'RTHOMPSON1' # default; will be overridden

    """
    Custom HTTP handler that serves static files and also
handles /api GET and POST requests,
calling an RPGLE program using itoolkit.
    """

    def call_rpg(self, method, req_data):

        """
        Call the RPGLE program 'WEBSERV' in library,
        passing in the HTTP method and request data as
parameters,
        and returning the RPG response.
```

```

Parameters:
    method (str): 'GET' or 'POST'
    req_data (str): JSON or text from the HTTP request

Returns:
    str: response string returned from RPG program
"""

# Initialize the toolkit object
itool = iToolkit()

# Add the library to the library list so the program can
be found
itool.add(iCmd('addliblible', f'ADDLIBLIBLE {self.LIBRARY}'))

# Add the RPG program call with 3 parameters:
# - method: 10-character input
# - request: 32000-character input
# - response: 32000-character output (blank string
placeholder)
itool.add(iPgm('pgmcall', 'WEBHNDLR')
        .addParm(iData('method', '10a', method))
        .addParm(iData('request', '32000a', req_data))
        .addParm(iData('response', '32000a', ' '))
        )

# Execute the RPG call
itool.call(itransport)

# Retrieve the output from the RPG program
result = itool.dict_out('pgmcall')

# Return the response value or fallback if missing
return result.get('response', 'No response')

# Handle HTTP GET requests to /api by calling the RPG
program
def do_GET(self):
    if self.path.startswith('/api'):

        # Parse query string into JSON string
        query = urlparse(self.path).query
        query_dict = parse_qs(query)

        # Check for shutdown command

```

```

        if query_dict.get('action', [''])[0].upper() ==
'SHUTDOWN':
            print("⚠ Shutdown requested via HTTP")

            def stop_server():
                print("▼ Shutting down server...")
                self.server.shutdown()

            threading.Thread(target=stop_server).start()

            self.send_response(200)
            self.send_header('Content-Type',
'application/json')
            self.end_headers()
            self.wfile.write(json.dumps({"status": "Server
shutting down"}).encode())
            return

        #Normal GET request processing
        query_json = json.dumps({k: v[0] for k, v in
query_dict.items()})

        request_data = query_json
        print("=== Incoming GET request ===")
        print(f"Query string: {query_json}")

        try:

            # Call the RPG program with method = 'GET'
            rpg_result = self.call_rpg('GET', request_data)
            print(f"Return Data: {rpg_result}")

            # Return the result to the client
            self.send_response(200)
            self.send_header('Content-Type',
'application/json')
            self.end_headers()
            self.wfile.write(rpg_result.encode())

        except Exception as e:
            print("=== Incoming GET request ===")
            print(f"Query string: {query_json}")

            print("=== RPG Response (raw) ===")
            print(rpg_result)

            print("ERROR during RPG call:", e)

```

```

        self.send_response(500)
        self.end_headers()
        self.wfile.write(json.dumps({"error":
str(e)}).encode())
    else:
        # Serve regular static files (like index.html, etc.)
        super().do_GET()

    # Handle HTTP POST requests to /api by calling the RPG
program
    def do_POST(self):
        if self.path.startswith('/api'):
            content_length = int(self.headers.get('Content-
Length', 0))
            content_type = self.headers.get('Content-Type', '')

            # Read the POST data
            post_data =
self.rfile.read(content_length).decode('utf-8')

            print("=== Incoming POST request ===")
            print(f"Content-Type: {content_type}")
            print(f"Raw POST data: {post_data}")

            try:
                # If content is JSON, parse it
                if 'application/json' in content_type:
                    # Parse JSON data
                    json_data = json.loads(post_data)
                    # Convert back to string for RPG program
                    request_data = json.dumps(json_data)
                else:
                    request_data = post_data

                print(f"Processed request data: {request_data}")

                # Call the RPG program with method = 'POST'
                rpg_result = self.call_rpg('POST', request_data)
                print(f"Return Data: {rpg_result}")

                # Return the result to the client
                self.send_response(200)
                self.send_header('Content-Type',
'application/json')
                self.end_headers()
                self.wfile.write(rpg_result.encode())

```

```

        except json.JSONDecodeError as e:
            print("ERROR parsing JSON:", e)
            self.send_response(400)
            self.send_header('Content-Type',
'application/json')
            self.end_headers()
            self.wfile.write(json.dumps({
                "success": False,
                "message": f"Invalid JSON format: {str(e)}"
            }).encode())

        except Exception as e:
            print("ERROR during RPG call:", e)
            self.send_response(500)
            self.send_header('Content-Type',
'application/json')
            self.end_headers()
            self.wfile.write(json.dumps({
                "success": False,
                "message": f"Error processing request:
{str(e)}"
            }).encode())

# -----
# START THE SERVER
# -----

if __name__ == '__main__':
    # Setup argument parser
    parser = argparse.ArgumentParser(description='Simple RPG
HTTP Server')
    parser.add_argument('--port', type=int, default=43001,
help='Port to listen on')
    parser.add_argument('--library', type=str,
default='RTHOMPSON1', help='Library to add to library list')

    args = parser.parse_args()

    # Save the library name globally
    RPGHTTPRequestHandler.LIBRARY = args.library

    # Start the HTTP server
    port = args.port
    server = ThreadingHTTPServer(('0.0.0.0', port),
RPGHTTPRequestHandler)
    print(f"Serving on http://localhost:{port} using library
{args.library}")
    print("Press Ctrl+C to stop the server")

```

```
try:
    server.serve_forever()
except KeyboardInterrupt:
    print("\n ⚠ Shutting down server gracefully...")
    server.shutdown()
    server.server_close()
    print(" 🛑 Server shutdown complete")
```

styles.css

Handles the formatting of our html.

```
.action-buttons {
    white-space: nowrap;
}

.action-buttons i {
    margin: 0 5px;
    cursor: pointer;
}

.fa-trash-alt:hover {
    color: red;
}

.fa-edit:hover {
    color: blue;
}

#notification {
    position: fixed;
    top: 60px;
    left: 50%;
    transform: translateX(-50%);
    min-width: 300px;
    max-width: 80%;
    padding: 15px;
    border-radius: 4px;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
    display: none;
    z-index: 900;
    /* Below modal by default */
}

#notification-message {
    display: block;
```

```
        white-space: pre-wrap;          /* Preserves newlines and
spaces */
        word-wrap: break-word;          /* Breaks long words */
        word-break: break-word;         /* Breaks words at wrap */
        overflow-wrap: break-word;      /* Ensures text wraps */
        margin-right: 20px;             /* Space for close button */
    }

#notification button {
    position: absolute;
    top: 5px;
    right: 5px;
    border: none;
    background: transparent;
    font-size: 20px;
    cursor: pointer;
    padding: 5px;
}

#notification.over-modal {
    z-index: 1100;
    top: 60px;
    left: 50%;
    transform: translateX(-50%);
}

#notification.success {
    background-color: #e8f5e9;
    border: 1px solid #66bb6a;
    color: #2e7d32;
}

#notification.error {
    background-color: #ffebee;
    border: 1px solid #ef5350;
    color: #c62828;
}

/* Navigation bar styles */
.navbar {
    overflow: hidden;
    background-color: #333;
}

.navbar a {
    float: left;
    font-size: 16px;
```

```
    color: white;
    text-align: center;
    padding: 14px 16px;
    text-decoration: none;
}

.dropdown {
    float: left;
    overflow: hidden;
}

.dropdown .dropbtn {
    font-size: 16px;
    border: none;
    outline: none;
    color: white;
    padding: 14px 16px;
    background-color: inherit;
    font-family: inherit;
    margin: 0;
}

.navbar a:hover,
.dropdown:hover .dropbtn {
    background-color: rgb(5, 33, 245);
}

.dropdown-content {
    display: none;
    position: absolute;
    background-color: #f9f9f9;
    min-width: 160px;
    box-shadow: 0px 8px 16px 0px rgba(0, 0, 0, 0.2);
    z-index: 1;
}

.dropdown-content a {
    float: none;
    color: black;
    padding: 12px 16px;
    text-decoration: none;
    display: block;
    text-align: left;
}

.dropdown-content a:hover {
    background-color: #ddd;
}
```

```
}

.dropdown:hover .dropdown-content {
  display: block;
}

/* Modal styles */
.modal {
  display: none;
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.4);
  z-index: 1000;
}

.modal-content {
  position: relative;
  background-color: #fff;
  margin: 15% auto;
  /* Increased from default to move down */
  padding: 20px;
  border: 1px solid #888;
  width: 90%;
  max-width: 400px;
  border-radius: 8px;
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
}

.close {
  float: right;
  font-size: 24px;
  font-weight: bold;
  cursor: pointer;
}

.close:hover {
  color: #666;
}

/* Form styling */
#productForm {
  margin-top: 20px;
}
```

```
#productForm label {
  display: inline-block;
  width: 120px;
  margin-bottom: 10px;
}

#productForm input {
  width: 200px;
  padding: 5px;
  margin-bottom: 10px;
}

#productForm button {
  margin-top: 15px;
  margin-right: 10px;
  padding: 8px 15px;
}
```

common.js

Container for most of our JavaScript code.

```
let notificationTimeout;

function showNotification(message, type) {
  const notification =
document.getElementById('notification');
  const messageElement =
document.getElementById('notification-message');
  const modal = document.getElementById('productModal');

  // Clear any existing timeout
  if (notificationTimeout) {
    clearTimeout(notificationTimeout);
  }

  // Remove previous type classes
  notification.classList.remove('success', 'error');

  // Add new type class
  notification.classList.add(type);

  // Position notification based on modal visibility
  if (modal.style.display === 'block') {
    notification.classList.add('over-modal');
  } else {
    notification.classList.remove('over-modal');
  }
}
```

```

        messageElement.textContent = message;
        notification.style.display = 'block';

        // Auto-hide after 10 seconds
        notificationTimeout = setTimeout(hideNotification, 10000);
    }

function hideNotification() {
    const notification =
document.getElementById('notification');
    notification.style.display = 'none';
    notification.classList.remove('over-modal');

    // Clear the timeout when manually closing
    if (notificationTimeout) {
        clearTimeout(notificationTimeout);
        notificationTimeout = null;
    }
}

function deleteProduct(productNumber) {
    if (confirm(`Are you sure you want to delete product:
${productNumber}?`)) {
        $.ajax({
            url: '/api',
            type: 'POST',
            contentType: 'application/json',
            data: JSON.stringify({
                app: 'product',
                action: 'delete',
                productnumber: productNumber
            }),
            success: function (response) {
                if (response.success) {
                    $('#myTable').DataTable().ajax.reload();
                    showNotification(response.message,
'success');
                } else {
                    showNotification(response.message, 'error');
                }
            },
            error: function (xhr, status, error) {
                showNotification('Error deleting product: ' +
error, 'error');
            }
        });
    }
}

```

```

    }
}

// Modal logic
function openProductModal(productNumber) {
    // Clear all form fields first
    $('#modalProductNumber').val('');
    $('#modalDescription').val('');
    $('#modalCost').val('');
    $('#modalUOM').val('');
    $('#modalCategory').val('');

    document.getElementById('productModal').style.display =
'block';
    if (productNumber) {
        // Edit mode
        $('#modalTitle').text('Edit Product');
        $('#modalProductNumber').prop('readonly', true);
        $.ajax({
            url:
`/api?app=product&action=getproduct&productnumber=${productNumber}`,
            type: 'GET',
            success: function (response) {
                // response.data contains the product record
                const product = response.data;

                $('#modalProductNumber').val(product.ProductNumber);
                $('#modalDescription').val(product.Description);
                $('#modalCost').val(product.Cost);
                $('#modalUOM').val(product.UnitOfMeasure);
                $('#modalCategory').val(product.Category);
            },
            error: function () {
                showNotification('Error retrieving product.',
'error');
                closeProductModal();
            }
        });
    } else {
        // Add mode
        $('#modalTitle').text('Add Product');
        $('#modalProductNumber').prop('readonly',
false).val('');
        $('#modalDescription').val('');
        $('#modalCost').val('');
        $('#modalUOM').val('');
    }
}

```

```

        $('#modalCategory').val('');
    }
}

function closeProductModal() {
    document.getElementById('productModal').style.display =
    'none';
}

function submitProductForm() {
    // Force cost to 0 if blank
    if ($('#modalCost').val() === '') {
        $('#modalCost').val(0);
    }
    let action = $('#modalTitle').text().startsWith('Edit') ?
    'updateproduct' : 'addproduct';
    const productData = {
        app: 'product',
        action: action,
        ProductNumber: $('#modalProductNumber').val(),
        Description: $('#modalDescription').val(),
        Cost: $('#modalCost').val(),
        UnitOfMeasure: $('#modalUOM').val(),
        Category: $('#modalCategory').val()
    };

    $.ajax({
        url: '/api',
        type: 'POST',
        contentType: 'application/json',
        data: JSON.stringify(productData),
        success: function (response) {
            if (response.success) {
                $('#myTable').DataTable().ajax.reload();
                showNotification(response.message, 'success');
                closeProductModal();
            } else {
                showNotification(response.message, 'error');
            }
        },
        error: function (xhr, status, error) {
            showNotification('Error saving product: ' + error,
            'error');
        }
    });
}

```

```

$(document).ready(function () {
    var table = $('#myTable').DataTable({
        ajax: {
            url: '/api?app=product&action=getproductlist',
            type: 'GET',
            dataSrc: 'data'
        },
        columns: [
            { 'data': 'ProductNumber' },
            { 'data': 'Description' },
            { 'data': 'Cost' },
            { 'data': 'UnitOfMeasure' },
            { 'data': 'Category' },
            {
                'data': null,
                'render': function (data, type, row) {
                    return `
                        <div class="action-buttons">
                            <i class="fas fa-edit"
title="Edit"
onclick="openProductModal('${row.ProductNumber}')"></i>
                            <i class="fas fa-trash-alt"
onclick="deleteProduct('${row.ProductNumber}')"
title="Delete"></i>
                        </div>
                    `;
                }
            }
        ],
        pageLength: 10,
        order: [[1, 'asc']]
    });
});

```

index.html

Our main page that displays our product table and handles the editing.

```

<!DOCTYPE html>
<html>

<head>
    <meta charset="UTF-8">
    <title>Product List</title>

```

```
<script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.m
in.js"></script>
<link href="https://cdn.datatables.net/v/dt/dt-
2.3.2/datatables.min.css" rel="stylesheet"
    integrity="sha384-
d76uxpdVr9QyCSR9vVSYd0AZeRzNUN8A4JVqUHBVXyGxZ+oOfrZVHC/1Y58mhyNg
" crossorigin="anonymous">
<script src="https://cdn.datatables.net/v/dt/dt-
2.3.2/datatables.min.js"
    integrity="sha384-
JRUjeYWWUG0171YFugrU0ksSC6CaWnl4Xzwp6mNjnnDh4hfFGRyYbEXwryGwLsEp
"
    crossorigin="anonymous"></script>
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.5.1/css/all.min.css">

<link rel="stylesheet" href="css/styles.css">
<script src="javascript/common.js"></script>
</head>

<body>
<!-- Product Modal -->
<div id="productModal" class="modal">
    <div class="modal-content">
        <span class="close"
onclick="closeProductModal()">&times;</span>
        <h2 id="modalTitle">Edit Product</h2>
        <form id="productForm">
            <label for="modalProductNumber">Product
Number:</label>
            <input type="text" id="modalProductNumber"
name="ProductNumber" readonly
            style="background:#eee;"><br><br>
            <label
for="modalDescription">Description:</label>
            <input type="text" id="modalDescription"
name="Description"><br><br>
            <label for="modalCost">Cost:</label>
            <input type="number" id="modalCost" name="Cost"
min=0 required value="0" step="0.01"><br><br>
            <label for="modalUOM">UOM:</label>
            <input type="text" id="modalUOM"
name="UnitOfMeasure"><br><br>
            <label for="modalCategory">Category:</label>
```

```

        <input type="text" id="modalCategory"
name="Category"><br><br>
        <button type="button"
onclick="submitProductForm()">Submit</button>
        <button type="button"
onclick="closeProductModal()">Cancel</button>
    </form>
</div>
</div>

<!-- Navigation Bar -->
<div class="navbar">
    <a href="/">Home</a>
    <div class="dropdown">
        <button class="drophbtn">Tools
            <i class="fa fa-caret-down"></i>
        </button>
        <div class="dropdown-content">
            <a
href="/api?app=product&action=getproductlist">Get Raw JSON
Product List</a>
            <a href="/api?action=shutdown">Shutdown Web
Server</a>
        </div>
    </div>
</div>

<br><br>

<div id="notification" style="display:none;">

    <span id="notification-message"></span>
    <button onclick="hideNotification()"
style="float:right;">&times;</button>
</div>
<br>
<div>
    <table id="myTable" class="display table table-striped
table-bordered" style="width:100%">
        <thead>
            <tr>
                <th>Product Number</th>
                <th>Description</th>
                <th>Cost</th>
                <th>UOM</th>
                <th>Category</th>
                <th>Actions</th>
            </tr>
        </thead>
    </table>
</div>

```

```

        </tr>
    </thead>
</table>
</div>
</body>

</html>

```

webhndlr.sqlrpgle

This program handles all the traffic from the web page to the DB2 database table.

```

**free
Ctl-Opt option(*nodebugio:*srcstmt:*nounref) Main(Main)
dftactgrp(*no)
    text('Web Server RPG Handler Program');

// The main procedure that will listen for incoming requests
Dcl-Proc Main;
Dcl-Pi *N;
    pMethod CHAR(10) const;
    pRequestData CHAR(32000) const;
    pResponseData CHAR(32000);
End-Pi;

Dcl-S vJsonString CHAR(32000);
Dcl-S vApp VARCHAR(50);
Dcl-S vAction VARCHAR(50);
// Dcl-S vRequestData SQLTYPE(CLOB:500000);

Exec Sql
    Set Option commit = *none;

    // If QueryString is blank display main page from the IFS
If pRequestData = '';
    pResponseData = '{"data": null}';
    Return;
Else;
    Exec Sql
        SELECT upper(jt.app)
            ,upper(jt.action)
        INTO :vApp
            ,:vAction
        FROM JSON_TABLE(trim(:pRequestData),
            'lax $'
            COLUMNS (
                app VARCHAR(50) PATH 'lax $.app',

```

```

        action VARCHAR(50) PATH 'lax $.action'
    )
    ) AS jt;

    If sqlcod <> 0;
        pResponseData = '{ "success": false, "message": "Invalid
JSON format" }';
        Return;
    EndIf;
EndIf;

If %upper(pMethod) = 'GET ';
    Select;
        When vApp = 'PRODUCT' and vAction = 'GETPRODUCTLIST';
            // Call the procedure to get the Product List in JSON
format
            vJsonString = GetProductList();
            pResponseData = vJsonString;
        When vApp = 'PRODUCT' and vAction = 'GETPRODUCT';
            // Call the procedure to get the Product List in JSON
format
            vJsonString = GetProduct(pRequestData);
            pResponseData = vJsonString;
        Other;
            pResponseData = '{"success": false, "message": "Unknown
Action Sent"}';
        EndSl;
    EndIf;

If %upper(pMethod) = 'POST';
    Select;
        When vApp = 'PRODUCT' and vAction = 'DELETE';
            vJsonString = DeleteProduct(pRequestData);
            pResponseData = vJsonString;
        When vApp = 'PRODUCT' And (vAction = 'UPDATEPRODUCT');
            vJsonString = UpdateProduct(pRequestData : vAction);
            pResponseData = vJsonString;
        Other;
            pResponseData = '{"success": false, "message":' +
                pMethod + '/' + vAction + ' Unknown Action Sent}';
        EndSl;
    EndIf;

End-Proc;

Dcl-Proc GetProductList;
    Dcl-Pi *N CHAR(32000);

```

```

End-Pi;

Dcl-S jsonString CHAR(32000);
Dcl-S jsonStringClob SQLTYPE(CLOB:32000);

Exec Sql
Select Cast(
    Json_Object(
        'data' Value (Json_Arrayagg(
            Json_Object(
                'ProductNumber' Value rtrim(product_number)
                , 'Description' Value Rtrim(description)
                , 'Cost' Value Cost
                , 'UnitOfMeasure' Value Rtrim(unit_of_measure)
                , 'Category' Value Rtrim(category)
            )
        ))
    ) As Clob(32000))
Into :jsonstringclob
From rthompson1.product_master;

JsonString = JsonStringClob_Data;
Return JsonString;
End-Proc;

Dcl-Proc GetProduct;
Dcl-Pi *N CHAR(32000);
    pRequestData CHAR(32000) const;
End-Pi;

Dcl-S jsonString CHAR(32000);
Dcl-S jsonStringClob SQLTYPE(CLOB:32000);
Dcl-S vProductNumber VARCHAR(20);

Exec Sql
    SELECT jt.productnumber
    INTO :vProductNumber
    FROM JSON_TABLE(trim(:pRequestData),
        'lax $'
        COLUMNS (
            productnumber VARCHAR(20) PATH 'lax $.productnumber'
        )
    ) AS jt;

If sqlcod <> 0;
    jsonString = '{"success": false, "message": "Invalid JSON
format"}';

```

```

        Return jsonString;
    EndIf;

    Exec Sql
        Select Cast(
            Json_Object(
                'data' Value
                Json_Object(
                    'ProductNumber' Value rtrim(product_number)
                    , 'Description' Value Rtrim(description)
                    , 'Cost' Value Cost
                    , 'UnitOfMeasure' Value Rtrim(unit_of_measure)
                    , 'Category' Value Rtrim(category)
                )
            )
            As CLOB(32000))
        Into :jsonstringclob
        From rthompson1.product_master
        Where product_number = :vProductNumber
        Fetch First Row Only;

    If sqlcod = 100;
        jsonString = '{"success": false, "message": "Product not
found"}';
        Return jsonString;
    EndIf;

    If sqlcod <> 0;
        jsonString = '{"success": false, "message": "Error Occurred
while retrieving product"}';
        Return jsonString;
    EndIf;

    JsonString = JsonStringClob_Data;
    Return JsonString;
End-Proc;

Dcl-Proc DeleteProduct;
    Dcl-Pi *N CHAR(32000);
        pRequestData CHAR(32000) const;
    End-Pi;

    Dcl-S vProductNumber VARCHAR(20);
    Dcl-S vJsonResponse VARCHAR(32000);

    Exec Sql
        SELECT jt.productnumber

```

```

        INTO :vProductNumber
        FROM JSON_TABLE(trim(:pRequestData),
            'lax $'
            COLUMNS (
                productnumber VARCHAR(20) PATH 'lax $.productnumber'
            )
        ) AS jt;

    If sqlcod <> 0;
        vJsonResponse = '{"success": false, "message": "Invalid JSON
format"}' ;
        Return vJsonResponse;
    EndIf;

    // Perform the delete operation
    Exec Sql
        DELETE FROM rthompson1.product_master
        WHERE product_number = :vProductNumber;

    If sqlcod = 0;
        vJsonResponse = '{"success": true, "message": "Successfully
deleted product: '
            + %trim(vProductNumber) + '" }';
    Else;
        vJsonResponse = '{"success": false, "message": "Failed to
delete product: '
            + %trim(vProductNumber) + '" }';
    EndIf;

    Return vJsonResponse;
End-Proc;

Dcl-Proc UpdateProduct;
    Dcl-Pi *N CHAR(32000);
        pRequestData CHAR(32000) const;
        pAction VARCHAR(50) const;
    End-Pi;

    Dcl-S vJsonResponse    VARCHAR(32000);
    Dcl-S vProductNumber   VARCHAR(20);
    Dcl-S vDescription     VARCHAR(100);
    Dcl-S vCost            PACKED(10:2);
    Dcl-S vUnitOfMeasure   VARCHAR(10);
    Dcl-S vCategory        VARCHAR(50);
    Dcl-S vExists          IND INZ(*Off);

    Exec Sql

```

```

SELECT jt.productnumber
      ,jt.description
      ,jt.cost
      ,jt.unitofmeasure
      ,jt.category
INTO :vProductNumber
      ,:vDescription
      ,:vCost
      ,:vUnitOfMeasure
      ,:vCategory
FROM JSON_TABLE(trim(:pRequestData),
  'lax $'
  COLUMNS (
    productnumber VARCHAR(20)    PATH 'lax
$.ProductNumber',
    description    VARCHAR(100)  PATH 'lax
$.Description',
    cost           DECIMAL(10,2)  PATH 'lax $.Cost',
    unitofmeasure  VARCHAR(10)    PATH 'lax
$.UnitOfMeasure',
    category       VARCHAR(10)    PATH 'lax $.Category'
  )
) AS jt;

If sqlcod <> 0;
  vJsonResponse = '{"success": false, "message": "Invalid JSON
format"}' ;
  Return vJsonResponse;
EndIf;

// Check to see if it exists
Exec Sql
  Select 1
    Into :vExists
  From rthompson1.product_master
  Where product_number = :vProductNumber
  Fetch First Row Only;

If sqlcod = 100 or Not vExists;
  vJsonResponse = '{"success": false, "message": "Product not
found"}';
  Return vJsonResponse;
EndIf;

// Edit the product records
If EditProduct(vProductNumber
              :vDescription

```

```

        :vCost
        :vUnitOfMeasure
        :vCategory
        :vJsonResponse
    ) = *On;

    Return vJsonResponse;
EndIf;

Exec Sql
    UPDATE rthompson1.product_master
    SET description = :vDescription
        ,cost = :vCost
        ,unit_of_measure = :vUnitOfMeasure
        ,category = :vCategory
    WHERE product_number = :vProductNumber;

If sqlcod = 0;
    vJsonResponse = '{"success": true, "message":
"Successfully updated product: '
        + %trim(vProductNumber) + '" }';
Else;
    vJsonResponse = '{"success": false, "message": "Failed to
update product: '
        + %trim(vProductNumber) + '" }';
EndIf;

Return vJsonResponse;
End-Proc;

Dcl-Proc EditProduct;
    Dcl-Pi *N IND; // Returns *On if there is an error
        pProductNumber VARCHAR(20) const;
        pDescription VARCHAR(100) const;
        pCost PACKED(10:2) const;
        pUnitOfMeasure VARCHAR(10) const;
        pCategory VARCHAR(50) const;
        pJsonResponse VARCHAR(32000);
    End-Pi;

    Dcl-S vErrorMessage VARCHAR(2000);

    If pDescription = '';
        vErrorMessage = 'Description cannot be blank ' + '\n';
    EndIf;

    If pCost < 0;

```

```

        vErrorMessage += 'Cost cannot be negative' + '\n';
    EndIf;

    If pUnitOfMeasure = '';
        vErrorMessage += 'Unit of Measure cannot be blank' + '\n';
    EndIf;

    If pCategory = '';
        vErrorMessage += 'Category cannot be blank' + '\n';
    EndIf;

    If vErrorMessage <> '';
        pJsonResponse = '{"success": false, "message": "' +
vErrorMessage + '"}';
        Return *On;
    EndIf;
    Return *Off;
End-Proc;

```

Now we have all the code and we have compiled the SQLRPGLE program (hopefully successfully) we are ready to give this a go.

Start the Web Server

Make sure the port you're using (e.g., 43001) is available on your IBMi.

From VS Code Open a PASE terminal by pressing (**Ctrl+Shift+J**) and selecting PASE and then type the following:

```

cd python/chapter8
python3 http_server.py --port 43001 --library RTHOMPSON1

```

It will take a few seconds to start the server. You will see a message in the terminal similar to this.

```

-bash-5.2$ python3 http_server.py --port 43001 --library RTHOMPSON1
Starting HTTP Server... Please wait...
Serving on http://localhost:43001 using library RTHOMPSON1

```

Now open a web browser and go to your IBMi name or IP address plus the port number.

```
http://myibmi:43001/
```

You should see web page open with a list of products being returned from our RPG program.

Home Tools ▾						
10 ▾ entries per page		Search:				
Product Number	Description	Cost	UOM	Category	Actions	
P0006	Coffee Mug	5.25	EACH	Kitchen		
P0009	Desk Lamp	18.75	EACH	Office		
P0020	Electric Kettle	24.5	EACH	Kitchen		
P0004	Laptop Backpack	45	EACH	Accessories		
P0016	LED Light Bulbs (6 Pack)	12.49	PACK	Home Improvement		
P0019	Mechanical Pencil Set (2 Pack)	4.75	PACK	Stationery		
P0007	Notebook - Lined	3.15	EACH	Stationery		
P0018	Notebook Cooling Pad	21.99	EACH	Electronics		
P0012	Pack of Sticky Notes	1.99	PACK	Stationery		
P0017	Phone Charger Cable	8.95	EACH	Electronics		
Showing 1 to 10 of 16 entries					« < 1 2 > »	

Now lets click the pencil icon for Edit. You will see the edit modal open up.

Home Tools ▾						
10 ▾ entries per page		Search:				
Product Number	Description	Cost	UOM	Category	Actions	
P0006	Coffee M			en		
P0009	Desk Lar			e		
P0020	Electric E			en		
P0004	Laptop B			ssories		
P0016	LED Ligh			e Improvement		
P0019	Mechanic			onery		
P0007	Notebook			onery		
P0018	Notebook			ronics		
P0012	Pack of S			onery		
P0017	Phone Ch			ronics		
Showing 1 to 10 of 16 entries					« < 1 2 > »	

×

Product Number:

Description:

Cost:

UOM:

Category:

Try making the description blank and click submit. You will see the error message display at the top in the notification area. After 10 seconds it will automatically disappear. There are many

ways to handle error messages but this seemed quick and simple. You can hit cancel to return to the main page or enter a new description and press Submit to save it. There are a few generic edits I put in this. Description, Unit of Measure and Category can not be blank. Also the cost can't be a negative number.

The screenshot shows a web application interface with a dark header containing 'Home' and 'Tools' with a dropdown arrow. Below the header, a red error message box states 'Description cannot be blank' with a close button. A modal window titled 'Edit Product' is open, displaying a form with the following fields: 'Product Number' (P0006), 'Description' (empty), 'Cost' (5.25), 'UOM' (EACH), and 'Category' (Kitchen). The modal has 'Submit' and 'Cancel' buttons at the bottom. In the background, a table lists products with columns for Product Number, Description, Cost, UOM, Category, and Actions. The first row shows P0006 with description 'Coffee Mug1'. The table footer indicates 'Showing 1 to 10 of 16 entries'.

Product Number	Description	Cost	UOM	Category	Actions
P0006	Coffee Mug1	5.25	EACH	Kitchen	[Edit] [Delete]

I changed the description to Coffee Mug1 and clicked submit. You should see the successful notification message and the screen refresh with the new description.

The screenshot shows the same web application interface after a successful update. A green notification message box states 'Successfully updated product: P0006' with a close button. The 'Edit Product' modal is no longer visible. The background table now shows the updated product: P0006 with description 'Coffee Mug1', cost 5.25, UOM EACH, and category Kitchen. The table footer indicates 'Showing 1 to 10 of 16 entries'.

Product Number	Description	Cost	UOM	Category	Actions
P0006	Coffee Mug1	5.25	EACH	Kitchen	[Edit] [Delete]

What's Next

With the dark knight defeated and the path secured, our hero meets a quiet figure waiting for him up ahead.

As you probably figured out by now we need to add the ability to create new records in our Product Master. You can sense this book is about to come to an end, are there any other challenges waiting ahead?